Bedrock Model Price Tracker: Spec

Oct 9, 2026 · @phil loatman

Goal and scope

A daily job tells us when a new model is available on Amazon Bedrock and how its input and output token prices compare with the models we use today.

In scope:

- Detect new Bedrock text models within one day of their appearing in our sources.
- Report on-demand list price per 1M input tokens and per 1M output tokens, in USD.
- Show the percent difference against a chosen baseline model.
- Detect list-price changes on models we already use.

Out of scope for version 1:

- Our actual spend or negotiated rates. Those come from AWS billing data.
- Batch, prompt-caching and provisioned-throughput prices.
- Quality, speed or token-efficiency comparisons between models.
- Providers other than Bedrock.

Data sources

The LiteLLM price file is the primary source; AWS is the check on it.

Source	Used for	Access	Notes
LiteLLM price file	New-model detection and prices	Public HTTPS GET, no key	Community-maintained, no update guarantee
Bedrock `ListFoundationModels`	Optional same-day detection of new models	AWS credentials in our account	Gives model IDs only, no prices
Bedrock pricing page	Manual confirmation of a new model's price	Browser	Source of truth for list prices

Fields read from each LiteLLM entry:

Field	Meaning
Entry key	Bedrock model ID, e.g. `anthropic.claude-sonnet-5` or `us.anthropic.claude-sonnet-5`
`litellm_provider`	Keep entries where this is `bedrock` or `bedrock_converse`
`mode`	Keep `chat` only
`input_cost_per_token`	USD per input token; multiply by 1,000,000
`output_cost_per_token`	USD per output token; multiply by 1,000,000
`deprecation_date`	Stored when present, shown in alerts

The same model appears under several keys: a bare ID plus prefixed IDs such as `us.`, `eu.`, `global.`, `au.` and `jp.`. Prices differ between them. In the file today, `us.anthropic.claude-sonnet-5` is $2.20 input and $11.00 output per 1M tokens, against $2.00 and $10.00 for the bare and `global.` IDs. The tracker follows one prefix, set in config, so each model is counted once.

Daily job

The job runs once a day on a scheduler and does the same seven steps each time.

1. **Fetch** the LiteLLM file. Retry three times, then fail the run.
2. **Validate** that it parses as JSON and still contains at least 80% of yesterday's Bedrock entries. If not, alert and stop without overwriting state.
3. **Filter** to Bedrock chat entries on the configured prefix, with both token prices present.
4. **Normalize** each entry to model ID, vendor, input price per 1M and output price per 1M.
5. **Diff** against the last good snapshot to produce three event types:
   - `new_model` : an ID not seen before
   - `price_change` : input or output price differs from the last snapshot
   - `removed` : an ID that has disappeared
6. **Compare** each new model against its baseline and compute the percent difference.
7. **Alert and save**: send one message per event, then write today's snapshot and append the events to the log.

The first run saves a snapshot and sends no alerts, so we are not notified about every existing model.

Optional step for same-day detection: call `ListFoundationModels` and alert on any model ID that LiteLLM does not yet have, marked "price pending".

Data model

The tracker keeps three things: a hand-edited config, a daily snapshot, and an append-only event log.

Config (edited by us)

Field	Example	Purpose
`prefix`	`us.`	Which variant of each model ID to follow
`watchlist`	`us.anthropic.claude-sonnet-4-6`	Models we use today
`baselines`	`anthropic.claude-sonnet-*` → `us.anthropic.claude-sonnet-4-6`	Which watchlist model a new model in that family is compared against
`alert_target`	Slack webhook or email address	Where alerts go

Snapshot (one row per model per day)

Field	Type
`snapshot_date`	date
`model_id`	string
`vendor`	string, the part of the ID before the first dot after the prefix
`input_per_1m_usd`	decimal
`output_per_1m_usd`	decimal
`deprecation_date`	date, nullable

Event log (one row per event)

Field	Type
`event_date`	date
`event_type`	`new_model`, `price_change` or `removed`
`model_id`	string
`baseline_model_id`	string, nullable
`input_per_1m_usd`, `output_per_1m_usd`	decimal
`input_pct_vs_baseline`, `output_pct_vs_baseline`	decimal, nullable
`previous_input_per_1m_usd`, `previous_output_per_1m_usd`	decimal, for `price_change` only

Comparison rules and alert format

Every new model is compared against one baseline, with input and output reported separately.

Baseline selection, in order:

1. The `baselines` entry whose pattern matches the new model ID.

2. If none matches, the watchlist model from the same vendor with the closest input price.
3. If the vendor has no watchlist model, no baseline. The alert shows prices only.

Percent difference:

```
\text{pct} = \frac{\text{new price} - \text{baseline price}}{\text{baseline price}} \times 100
```

Negative means cheaper. Round to one decimal place. Input and output are never blended into a single number, because our input-to-output ratio varies by workload.

Example alert, using prices in the LiteLLM file today:

```
New on Bedrock: us.anthropic.claude-sonnet-5
Baseline:       us.anthropic.claude-sonnet-4-6

          New      Baseline   Difference
Input     $2.20    $3.30      -33.3%
Output    $11.00   $16.50     -33.3%
(USD per 1M tokens, on-demand list price)

Source: LiteLLM price file. Confirm at aws.amazon.com/bedrock/pricing
```

A `price_change` alert uses the same layout with the previous price in place of the baseline. A `removed` alert is one line and is sent only for watchlist models.

Failure handling and known limits

A failed run never overwrites the last good snapshot, so one bad day cannot produce a flood of false alerts.

Situation	Behavior
Fetch fails after retries	Alert "tracker run failed" and keep yesterday's snapshot
File fails validation	Same as above
Entry has no input or output token price	Skip it and count it in the run summary

Situation	Behavior
Watchlist model missing from the file	Alert once, then stay quiet until it returns
Same event detected twice	De-duplicate on model ID, event type and prices
No run for 2 days	A separate heartbeat check alerts

Known limits:

- **Lag.** LiteLLM has no update schedule. A new model can appear days after AWS launches it. The optional `ListFoundationModels` step covers detection but not price.
- **Accuracy.** The file is community-maintained. Confirm a new model's price on the AWS pricing page before acting on it.
- **List price only.** Negotiated rates, batch discounts and caching are not reflected.
- **Tiered pricing.** Some models charge more above 200K input tokens. Version 1 reports the base tier only.
- **Price is not cost.** A cheaper model can use more tokens for the same task, so the percentage is a starting point for a cost estimate.

Open decisions

Six choices are needed before the build starts.

- [ ] Which model ID prefix do we call Bedrock with (`us.`, `global.`, bare, other)? The spec assumes `us.`.
- [ ] Which models go on the watchlist, and which is the baseline for each family?
- [ ] Where do alerts go: Slack channel, email, or a file only?
- [ ] Where does the job run and store state: a GitHub Actions workflow committing CSVs to a repo, or a Lambda with S3?
- [ ] Do we add the `ListFoundationModels` step in version 1, which needs AWS credentials, or start with LiteLLM only?
- [ ] Does the org allow a scheduled job to fetch a public file from GitHub? Confirm with whoever owns that policy.

Sources

- LiteLLM repository

- LiteLLM price file, raw, read 2026-10-09 for the field names and example prices above
- Amazon Bedrock pricing